INDIAN PEDIATRICS
AI ASSISTANT

ROADMAP

From product idea to a testable proof of concept

A teaching guide to the sequence, decisions, gates, and outputs for a reusable medical-journal RAG assistant.

Teaching manual installment 1.0 | October 2026

Project: Reusable medical-journal literature assistant
POC corpus: Indian Pediatrics, January 2025, Volume 62, Issue 1
Audience: AI students and developers learning to plan and build a grounded literature assistant

How to use this installment
These are the first chapters of a cumulative teaching manual. They explain decisions made so far and provide a reproducible way to understand them. The full 300+ page reference is intended to grow as later roadmap phases are completed.

Document control and scope

This roadmap follows the seven-phase outline in *Ver 1 - Complete Implementation Guide for POC*. It preserves the Phase 1-7 names and the guide's numbered actions 1-43. The Master Guide supplies the broader product principles: evidence discovery, not diagnosis; citations from real sources; staged scope; article-type awareness; and measurable quality review.

Current project status
Phase 1 (Development Foundation) and Phase 2 (Data Acquisition / Define and Acquire the POC Corpus) have passed their project checkpoints. The selected corpus is the January 2025 issue. Phase 3 is the next implementation phase. A passing setup or corpus checkpoint is not proof of clinical accuracy.

Contents at a glance

- 1. The project: intended user, problem, and boundaries
- 2. Design principles that shape the sequence
- 3. Roadmap at a glance: Phases 1-7
- 4. Detailed numbered implementation sequence (1-43)
- 5. Phase gates, evidence, and change control
- 6. Reuse across journals and future expansion
- 7. Risks, anti-patterns, and learning approach
- 8. References and glossary

1. The project: intended user, problem, and boundaries

The problem is information discovery across a large journal archive. A pediatric professional may know the clinical topic but not the exact article title, author, issue, or keyword. A literature assistant should help locate, compare, and understand what the selected journal has published.

The assistant is designed as a domain-specific knowledge retrieval system. A question is interpreted, relevant journal material is retrieved, evidence is summarized, and the answer points back to articles. This is a RAG pattern: the model receives retrieved source passages at answer time rather than relying only on its pretrained memory.

The initial POC is not an AI doctor. It must not diagnose, prescribe, replace clinical judgement, or imply evidence that is not in the selected corpus. It is a journal-literature navigation and synthesis prototype.

1.1 Users and tasks

Primary users include pediatricians, residents, researchers, medical educators, and students. The initial tasks are topic search, evidence summary, article comparison, type-aware filtering, author or date lookup, and a safe no-answer response when the corpus is insufficient.

1.2 POC scope

The current POC is intentionally small: one issue, January 2025, Volume 62, Issue 1 of Indian Pediatrics, with the supplied local PDFs and a structured catalogue. This gives us different article forms without taking on the risks of bulk archive acquisition.

2. Design principles that shape the sequence

These principles explain why later work depends on earlier preparation. The sequence is a quality-control strategy, not bureaucracy.

Principle	Why it matters	Practical rule
Evidence before fluency	A fluent answer can still be unsupported.	Keep retrieved passages and citations available for every answer.
Metadata plus text	Semantic matching alone cannot reliably filter by author, year, volume, section, or type.	Model the article first; attach article metadata to every later text chunk.
Preserve source files	Processing errors must be recoverable and auditable.	Keep raw originals unchanged; create derived text elsewhere.
Small, staged corpus	A huge archive magnifies extraction, permission, and QA problems.	Prove the pipeline on a bounded issue before expansion.
Modular and reusable	A second journal should require new configuration and mapping, not a rewrite.	Separate shared processing logic from journal-specific settings and source adapters.
Explicit uncertainty	The system must be able to say the corpus does not answer a question.	Define a no-answer threshold and evaluate it intentionally.
Build only what is needed	Premature agent orchestration and infrastructure increase failure points.	Start with one transparent retrieval and generation workflow.

3. Roadmap at a glance

The Implementation Guide organizes the work into seven phases. Numbering below stays aligned to that guide. Names have been retained, with Step 1 and Step 2 also described using the labels adopted during this project.

Phase	Phase name	Main outcome	Gate to next phase
Phase 1	Development Foundation / Set Up the Development Foundation	Reproducible local project, journal-neutral structure, settings, Git, startup and readiness check.	Step 1 checker passes; no unexplained failures.
Phase 2	Data Acquisition / Define and Acquire the POC Corpus	Issue scope, article catalogue, file manifest, exclusion decisions and corpus checkpoint.	Step 2 checker passes; source/use scope recorded.
Phase 3	Data Processing	Extract and clean PDF/HTML text; preserve structure and page provenance; inspect quality.	Representative files pass extraction and quality review.
Phase 4	RAG	Chunk, embed, index, retrieve, generate grounded answers and cite sources.	Retrieval, citation, and no-answer behavior demonstrated.
Phase 5	Application	Provide a basic interface and core search, answer, source, comparison and filtering workflows.	User can complete intended workflows reliably.
Phase 6	Validation	Build golden questions and evaluate retrieval, answers, citations, and safety.	Known defects fixed or recorded against stated targets.
Phase 7	Demonstration	Select demo questions, polish the prototype, prepare script and technical evidence.	POC V1 demo package is reviewable; limits disclosed.

3.1 Dependency chain

The primary dependency is: requirements and project foundation -> controlled data -> metadata and quality -> retrieval -> grounded answers and citations -> user workflows -> validation -> demonstration. A later stage may reveal an earlier defect; the process must allow a controlled return to fix it.

4. Detailed numbered implementation sequence (1-43)

Phase 1 - Development Foundation

1. Set up Python environment. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

2. Create Git repository. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

3. Create reusable folder structure. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

4. Install only the foundation packages needed. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

5. Create safe environment-variable template. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

6. Create application and journal configuration. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

7. Provide optional LLM connection test when service credentials are approved. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

8. Provide optional embedding connection test when service credentials are approved. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

Phase 2 - Data Acquisition

9. Study January 2025 issue structure. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

10. Create article_metadata.csv. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

11. Identify usable articles and content roles. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

12. Collect accessible article documents. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

13. Store raw documents with traceable names and manifest. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

Phase 3 - Data Processing

14. Extract PDF or HTML text. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

15. Clean extraction artifacts without changing meaning. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

16. Attach validated metadata. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

17. Inspect extraction and metadata quality. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

Phase 4 - RAG

18. Chunk documents by meaningful section. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

19. Create embeddings. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

20. Build and persist the vector/search index. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

21. Implement retrieval. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

22. Add hybrid keyword and semantic search. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

23. Build grounded answer generation. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

24. Create verified source citations. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

25. Implement abstention / no-answer logic. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

Phase 5 - Application

26. Build Streamlit POC interface. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

27. Implement natural-language search. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

28. Render the answer and key findings. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

29. Show relevant source articles and links. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

30. Support comparison workflow. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

31. Filter by article type. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

Phase 6 - Validation

32. Create the Golden Test Set. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

33. Evaluate retrieval. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

34. Evaluate answer correctness and grounding. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

35. Evaluate citation validity. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

36. Evaluate hallucination and no-answer behavior. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

37. Fix and retest defects. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

Phase 7 - Demonstration

38. Select 5-7 demonstration questions. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

39. Polish interface for the intended audience. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

40. Prepare a reproducible demonstration script. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

41. Prepare the technical manual. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

42. Prepare the POC test report. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

43. Declare POC V1 ready only against explicit criteria. Define the input, execute the task, capture the output and verify it before relying on it in the next step.

5. Phase gates, evidence, and change control

A gate is a documented decision that the next phase has the prerequisites it needs. A gate does not mean that the whole product is correct. Record what was checked, the result, warnings, unresolved assumptions, and the exact follow-up owner/action.

Gate	Evidence to retain	Stop and fix when...
Phase 1	Environment notes, configuration, folder tree, Git root, startup output, checker result.	The app cannot start predictably, required paths/config fail, or secret handling is unsafe.
Phase 2	Issue source URL, catalogue, file manifest, scope/exclusion notes, acquisition basis, checker output.	Files are untraceable, article IDs duplicate, required fields fail, or scope/access basis is unclear.
Phase 3	Extracted text samples, page/section mapping, issue log, QA decisions.	Titles/pages are mismatched, text is missing or corrupted, or article boundaries are wrong.
Phase 4	Retrieval examples, chunks and source links, citation checks, answer/no-answer logs.	Answers cannot be traced to retrieved evidence or the system invents support.
Phase 5-7	Workflow walkthrough, Golden Test Set results, demo checklist/report.	Critical defects remain hidden or claims exceed evaluation evidence.

5.1 Decision log template

Field	Record
Decision ID and date	Unique entry and date of change.
Question	What choice had to be made?
Options considered	Include viable alternatives, not every imaginable option.
Choice and rationale	What was selected and why?
Trade-offs / risks	What capability or assurance is limited?
Evidence / approval	Link to source, checker, user decision, or permission review.
Revisit trigger	What new fact would justify changing the decision?

6. Reuse across journals and future expansion

The product is being designed as a reusable template. The shared application should not assume every journal has the same website, PDF layout, section vocabulary, publication cadence, or reuse terms. The first journal-specific configuration describes identity and source URLs; future journal onboarding adds a source adapter or parsing rules when needed.

Shared across journals	Configured per journal	Validated per new journal
Article/chunk data model, provenance fields, checker patterns, retrieval interfaces, citation representation, safety behavior.	Journal ID/name, issue URLs, section mappings, publication calendar, access status, corpus selection.	Website structure, PDF layouts, article-type vocabulary, author formatting, access conditions, extraction quality and search behavior.

6.1 Portability test

A genuinely reusable system can add a second journal without editing Indian Pediatrics assumptions throughout the core. The test is: create a journal configuration from the example template, acquire a small permitted corpus, map that journal's sections, pass data checks, and run the same retrieval/evaluation contract. If core code requires many journal-name conditionals, separate the journal-specific rule into configuration or an adapter.

7. Risks, anti-patterns, and learning approach

The common temptation is to begin with a chat screen because it feels like visible progress. In this project, the less visible work - scope, source identity, metadata, text quality and evaluation - determines whether answers can be trusted.

Avoid	Why it fails	Safer practice
Download the entire archive at once	Scale hides rights issues and multiplies OCR/metadata defects.	Start with one issue and expand only after measured success.
Treat every PDF as a clinical paper	Notices, news, duplicates and clippings do not have the same evidence value.	Assign content roles and retrieval eligibility.
Use filenames as IDs	Page-number filenames are ambiguous and changeable.	Assign persistent article IDs; store original filenames separately.
Use embeddings as the complete search solution	Exact author/title/year searches and filters can fail in pure vector search.	Combine semantic, lexical, metadata filters and reranking.
Let the language model create citations	The model may fabricate bibliographic details.	Build citations from retrieved record fields and verify links.
Add autonomous agents before a reliable baseline	More moving parts complicate debugging and evaluation.	First deliver one inspectable RAG workflow.
Claim clinical validation from a demo	A small journal corpus and informal review do not establish clinical safety.	Call it a literature POC; document scope and evaluation limits.

8. References and glossary

The roadmap is aligned to the project documents and current implementation records listed below. Where the old planning documents differ from the project as actually built, this manual describes the current project state and calls out the difference rather than silently copying stale values.

- Ver 1 - Master Guide for POC.pdf: product rationale, scope, RAG principles, quality targets, and the 43-step high-level flow.
- Ver 1 - Complete Implementation Guide for POC.pdf: seven implementation phases and 43 numbered implementation actions.

- Ver 1 - Implementation - Step 1.pdf: detailed legacy environment setup example; some paths, Python versions, and API-check expectations differ from the current project.
- Current Project README.md, config files, scripts/check_step1.py, scripts/check_step2.py, and Step 2 metadata/checkpoint records.

Glossary

Term	Plain-language meaning
RAG	Retrieval-Augmented Generation: retrieve source evidence and provide it to a language model for a grounded response.
Corpus	The bounded set of source documents selected for an application or experiment.
Metadata	Structured facts describing an article, such as title, author, date, issue, pages and type.
Gate	A checkpoint that confirms prerequisites and makes unresolved issues visible.
Abstention	A deliberate response that says evidence is insufficient or absent.
Golden Test Set	Questions with expected relevant sources/findings, used to test system behavior.
POC	Proof of Concept: a limited demonstration that an idea can work under stated conditions.